

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Отчет о научно-исследовательской работе
«Технологии параллельного программирования в
операционных системах Linux»

Студент,
группы 5130201/30003

_____ Мартыненко А. П.

Преподаватель

_____ Чуватов М.В.

«_____» _____ 2025г.

Санкт-Петербург, 2025

Реферат

Отчет 34 с., 12 рис., 5 источн., 3 прил.

ПАРАЛЛЕЛЬНОЕ ПРОГРАММИРОВАНИЕ, ТЕХНОЛОГИЯ OPENMP, ТЕХНОЛОГИЯ OPENMPI, LINUX СИСТЕМЫ, ОСТОВ ГРАФА, КОД ПРЮФЕРА

Объектом исследования являются технологии параллельного программирования в ОС на основе ядра Linux.

Цель работы - изучение настройки сетей и работа с ОС Linux, ознакомление с средой виртуализации VirtualBox, изучение технологий параллельного программирования OpenMP и OpenMPI в системах Linux, применение этих технологий для параллелизации лабораторной работы по теории графов.

В процессе работы были установлены среда виртуализации VirtualBox и дистрибутив CentOS Stream 9. Была проведена настройка сети из нескольких узлов, изучены основные возможности взаимодействия с ОС Linux и технологиями параллельного программирования. Подготовлена среда разработки и выполнена параллелизация лабораторной работы по теории графов.

В результате исследования была разработана программа на языке C++, которая использует технологии OpenMP и OpenMPI. Приложение может выполняться на трех вычислительных узлах параллельно, каждый из которых реализует многопоточные вычисления.

Содержание

Термины и определения	4
Перечень сокращений и обозначений	5
Введение	6
1 Основная часть	7
1.1 Установка и настройка среды виртуализации	7
1.1.1 Настройка hosts	8
1.1.2 Настройка SSH подключения	9
1.1.3 Настройка буфера обмена с внешней средой	10
1.1.4 Настройка OpenMP	11
1.1.5 Настройка OpenMPI	12
1.2 Применение технологий параллелизации к исходной программе	14
1.2.1 Применение OpenMP	14
1.2.2 Применение OpenMPI	15
2 Результаты работы программы	18
Заключение	19
Список использованной литературы	20
Приложение А «Исходный код программы: файл main.cpp»	21
Приложение А «Исходный код программы: файл graph.cpp»	23
Приложение А «Исходный код программы: файл graph.h»	33

Термины и определения

В настоящем отчете о НИР применяют следующие термины с соответствующими определениями:

1. Виртуальная машина - это программная имитация физического компьютера, которая позволяет запускать на нем операционную систему.
2. Операционная система - это комплекс программ, управляющих аппаратными ресурсами компьютера и обеспечивающее интерфейс для взаимодействия пользователя и оборудования.
3. Сеть NAT - это технология трансляции сетевых адресов, которая позволяет переводить IP-адреса между локальной и внешней сетью.
4. IP - это сетевой протокол, который определяет правила адресации и маршрутизации данных между устройствами в компьютерной сети.
5. IP-адрес - уникальный числовой идентификатор компьютера или другого сетевого устройства для обеспечения адресации и связи между устройствами.
6. DHCP - это сетевой протокол, который автоматически назначает IP-адреса и другие параметры сети компьютерам в локальной сети.
7. SSH - это сетевой протокол для безопасного удаленного доступа к компьютерам по зашифрованному соединению, позволяющий управлять системой, передавать файлы и запускать команды.
8. Домен - это имя в системе доменных имен DNS, которое используется для упрощения доступа к ресурсам в сети.
9. Open MP - это набор директив компилятора, библиотечных процедур и переменных окружения, которые предназначены для программирования многопоточных приложений на многопроцессорных системах с общей памятью.
10. Open MPI - это библиотека, предназначенная для разработки параллельных приложений на системах с распределенной памятью.
11. Код Прюфера - это способ кодирования остовного дерева с n вершинами в виде последовательности из $n-2$ чисел, однозначно соответствующей дереву.
12. ISO-образ — это файл, содержащий точную копию (образ) файловой системы оптического диска.

Перечень сокращений и обозначений

В настоящем отчете о НИР применяют следующие сокращения и обозначения:

ВМ - виртуальная машина.

ОС - операционная система.

NAT - Network Address Translation.

IP - Internet Protocol.

DHCP - Dynamic Host Configuration Protocol.

SSH - Secure Shell.

MP - Muplti-Processing.

MPI - Message Passing Interface.

ISO - International organization for standardization.

DNS - Domain Name System.

Введение

В рамках научно-исследовательской работы необходимо было ознакомиться с технологиями параллельного программирования в ОС на базе ядра Linux. Системы Linux считаются одними из самых стабильных и безопасных, поэтому широко используются в серверных решениях. В данной работе используется дистрибутив CentOS Stream 9, а также технологии OpenMP и OpenMPI.

В качестве среды виртуализации использовалась программа VirtualBox. С ее помощью можно создать виртуальные машины различных операционных систем и симулировать работу простейшего суперкомпьютера с несколькими узлами. Данное приложение имеет русифицированный интерфейс.

В соответствии с выданным вариантом был использован дистрибутив CentOS Stream, сеть 172.16.120.0/24.

Выполнение работы можно разбить на следующие этапы:

- 1) установка и настройка среды виртуализации VirtualBox, создание трех виртуальных машин;
- 2) настройка локальной сети и SSH-подключения между машинами;
- 3) установка и настройка ПО для работы с OpenMP и OpenMPI на виртуальных машинах, запуск тестовых программ;
- 4) параллелизация лабораторной работы по теории графов;
- 5) запуск программы на виртуальных машинах.

1 Основная часть

1.1 Установка и настройка среды виртуализации

Для создания приложения с поддержкой многопоточности на ОС Linux была использована программа виртуализации Oracle VirtualBox, которую можно скачать с официального сайта. В качестве дистрибутива для работы в операционной среде Linux был скачан ISO-образ CentOS Stream 9 с официального сайта для архитектуры x86_64.

В начале работы необходимо настроить сеть. Для этого нужно открыть вкладку «Файл», перейти в панель «Инструменты», затем «Менеджер сетей», вкладка «Сети NAT» и нажать «Создать». IPv4 префикс нужно указать в соответствии с вариантом (172.16.120.0/24), а также выключить автоматическую настройку сетевого интерфейса DHCP.

Следующим шагом необходимо создать виртуальную машину:

1. В интерфейсе VirtualBox нажать «Создать», указать имя виртуальной машины, место ее установки и путь до ранее скачанного ISO-образа дистрибутива CentOS Stream 9. Также нужно отметить пункт «Пропустить автоматическую установку».
2. Для каждой машины нужно выделить ресурсы: объем оперативной памяти 2048 МБ, количество процессоров - 2, размер виртуального жесткого диска 20 ГБ.
3. В настройках созданной виртуальной машины во вкладке «Сеть» для «Адаптера 1» необходимо выбрать тип подключения «Сеть NAT» с именем ранее созданной сети. Если включены остальные сетевые адаптеры, то их следует выключить.

После создания виртуальной машины ее нужно запустить и установить саму операционную систему. В большинстве вкладок можно оставить стандартные параметры, самостоятельно нужно настроить следующие вкладки:

1. Так как DHCP отключен, сеть нужно настраивать вручную. В полученном варианте сеть 172.16.120.0/24, следовательно, следующий адрес (172.16.120.1) является адресом шлюза и адресом сервера DNS. Вторым адресом тоже не рекомендуется использовать для IP виртуальных машин, так как могут возникнуть проблемы с подключением извне. Для первой ВМ назначим адрес 172.16.120.10, для второй - 172.16.120.11 и так далее.
2. Выбираем имя машины в сети (имена должны быть уникальными) и домен (например, hrc). Имя домена для всех виртуальных машин должно быть одинаковым.

3. После этого создаем обычного пользователя и задаем пароль. Желательно выбрать везде одинаковое имя пользователя и пароли, чтобы упростить запуск программ. Пароль для суперпользователя задавать необязательно. В этом случае обычные пользователи с помощью команды `sudo` смогут выполнять действия, требующие привилегий.
4. При установке системы устанавливаем Server без графического окружения, стандартные системные утилиты и SSH-сервер.

После всего этого у нас будет виртуальная машина с установленной ОС и настроенной сетью.

Для простоты использования остальные виртуальные машины можно не создавать заново, а клонировать, сгенерировав новые MAC адреса. В этом случае IP адрес и имя не изменяется, и для дальнейшего обращения их нужно поменять. Для этого используется утилита `nmtui`, где выбирается нужное соединение. Внутри вкладки «Редактировать соединение» необходимо указать новый IP-адрес и новое имя VM.

1.1.1 Настройка hosts

Для обращения к разным узлам виртуальной сети не по IP адресам, а по именам, необходимо настроить разрешение имен. Для этого нужно записать в файл `/etc/hosts` соответствия имен и адресов. Этот файл позволяет разрешать имена узлов без настройки службы DNS. Эту процедуру необходимо проделать на каждом узле сети. Открыть файл с конфигурацией можно по команде `sudo nano /etc/hosts`. Проверить связь между узлами можно командой `ping host_name`.

```
127.0.0.1    localhost localhost.localdomain localhost4 localhost4.localdomain4
::1         localhost localhost.localdomain localhost6 localhost6.localdomain6
172.16.120.10 node1.chp node1
172.16.120.11 node2.chp node2
172.16.120.12 node3.chp node3
172.16.120.13 node4.chp node4
```

Рис. 1. Настроенный файл `/etc/hosts`


```

[root@node2 ~]# ping node1
PING node1.chp (172.16.120.10) 56(84) bytes of data.
64 bytes from node1.chp (172.16.120.10): icmp_seq=1 ttl=64 time=1.12 ms
64 bytes from node1.chp (172.16.120.10): icmp_seq=2 ttl=64 time=0.357 ms
64 bytes from node1.chp (172.16.120.10): icmp_seq=3 ttl=64 time=0.419 ms
64 bytes from node1.chp (172.16.120.10): icmp_seq=4 ttl=64 time=0.425 ms
^C
--- node1.chp ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3143ms
rtt min/avg/max/mdev = 0.357/0.580/1.122/0.313 ms
[root@node2 ~]# ping node2
PING node2.chp (172.16.120.11) 56(84) bytes of data.
64 bytes from node2.chp (172.16.120.11): icmp_seq=1 ttl=64 time=0.059 ms
64 bytes from node2.chp (172.16.120.11): icmp_seq=2 ttl=64 time=0.060 ms
64 bytes from node2.chp (172.16.120.11): icmp_seq=3 ttl=64 time=0.058 ms
64 bytes from node2.chp (172.16.120.11): icmp_seq=4 ttl=64 time=0.059 ms
^C
--- node2.chp ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3080ms
rtt min/avg/max/mdev = 0.058/0.059/0.060/0.000 ms

```

Рис. 2. Проверка связи между узлами

1.1.2 Настройка SSH подключения

SSH - это криптографический сетевой протокол, который обеспечивает безопасное удаленное управление операционной системой. С его помощью можно управлять виртуальными машинами, передавать или копировать файлы по зашифрованным каналам.

Авторизация по SSH может выполняться с использованием парольной аутентификации или аутентификации по ключу. В данной работе настраивается вход по ключу.

При настройке ОС SSH-сервер был установлен на каждой виртуальной машине. Для генерации ключа нужно выполнить команду **ssh-keygen -A** на каждой машине. Все параметры оставляем по умолчанию: пароль для закрытого ключа не устанавливается, ключевая пара сохраняется в каталоге `/.ssh/`.

Для размещения открытого ключа на удаленных узлах необходимо ввести команду **ssh-copy-id user@host_name**, где `user` - имя пользователя на удаленном хосте, `host_name` - адрес удаленного хоста, к которому подключается данная виртуальная машина. Если имя пользователя одинаковое на всех узлах, его можно опустить и вводить команду в виде **ssh-copy-id host_name**. В случае, если нужно перезаписать существующие записи ключа, к команде нужно добавить **-f (force)**: **ssh-copy-id -f user@host_name**.

После успешной настройки SSH-подключения ко всем машинам, доступ из одной машине к другой должен осуществляться без пароля. Для проверки можно использовать команду **ssh user@host_name**. Например, на картинке 3 представлено подключение из первого узла ко второму, из второго - к третьему. Все вариации подключений работают успешно.

```

[root@node1 ~]# ssh root@node2
Warning: your password will expire in 0 days.
Warning: your password will expire in 0 days.
Activate the web console with: systemctl enable --now cockpit.socket

Last login: Fri Jun 20 12:07:12 2025 from 172.16.120.10
[root@node2 ~]# ssh root@node3
Warning: your password will expire in 0 days.
Warning: your password will expire in 0 days.
Activate the web console with: systemctl enable --now cockpit.socket

Last login: Fri Jun 20 12:07:37 2025 from 172.16.120.13
[root@node3 ~]# ssh root@node4

```

Рис. 3. Подключение по SSH

1.1.3 Настройка буфера обмена с внешней средой

Для выполнения работы необходимо иметь возможность пересылать файлы из основной операционной системы на виртуальные машины. Для этого нужно перейти во вкладку «Инструменты», выбрать «Менеджер сетей», «Сети NAT» и «Проброс портов». Проброс портов настраивается для каждой машины отдельно. В поле «Адрес хоста» везде указываем стандартный локальный адрес - 127.0.0.1. «Порт хоста» необходимо указать больше 4096, чтобы не конфликтовать с привилегированными хостами (в данном случае указываются 40301-40304). «Адрес гостя» - это IPv4 адрес виртуальной машины, а «порт гостя» по умолчанию 22.

Имя	Протокол	Адрес хоста	Порт хоста	Адрес гостя	Порт гостя
Rule 1	TCP	127.0.0.1	40301	172.16.120.10	22
Rule 2	TCP	127.0.0.1	40302	172.16.120.11	22
Rule 3	TCP	127.0.0.1	40303	172.16.120.12	22

Рис. 4. Настроенный проброс портов

Проверка корректности проброса портов осуществляется с помощью команды `ssh user@local_host -p port`. Если все выполнено верно, то из командной строки основной системы получится зайти на виртуальную машину.

```

C:\Users\marty>ssh root@127.0.0.1 -p 40301
root@127.0.0.1's password:
Warning: your password will expire in 0 days.
Warning: your password will expire in 0 days.
Activate the web console with: systemctl enable --now cockpit.socket

Last login: Sat Jun 21 12:58:14 2025
[root@node1 ~]#

```

Рис. 5. Подключение с Windows cmd

Для отправки файлов на виртуальную машину с ОС используется команда `scp -P host_port source_path user@local_host:dest_path`.

```
C:\Users\marty>scp -P 40301 -r "C:\Users\marty\source\repos\lab_practice" root@127.0.0.1:~
root@127.0.0.1's password:
Warning: your password will expire in 0 days.
functions.h          100% 4424      2.1MB/s   00:00
graph.cpp            100% 24KB      15.3MB/s   00:00
graph.h              100% 5018      4.7MB/s   00:00
hamCycles.txt        100% 400      380.4KB/s  00:00
main.cpp             100% 18KB      180.0KB/s  00:00
```

Рис. 6. Отправка данных

1.1.4 Настройка OpenMP

OpenMP (Open Multi-Processing) - это набор директив компилятора, библиотечных процедур и переменных окружения, которые предназначены для программирования многопоточных приложений на многопроцессорных системах с общей памятью.

Принцип работы OpenMP заключается в использовании модели параллельного выполнения «ветвление-слияние», где изначально программа выполняется в единственном главном потоке. При обнаружении специальных директив компилятора `#pragma omp parallel` главный поток создает группу новых потоков, которые начинают параллельное выполнение кода (схема на рисунке 7). При этом уровней вложенности таких параллельных секций может быть несколько, а количество параллельных потоков задаваться вручную. Также можно ограничивать области видимости потоков (`private`, `shared`, `reduction` и другие).

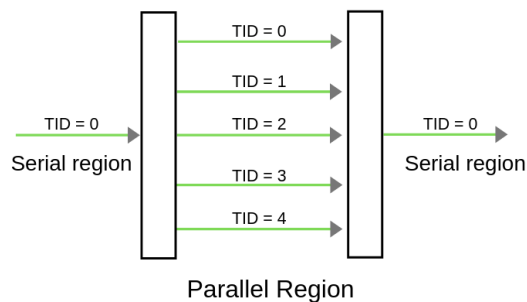


Рис. 7. Схема работы OpenMP

OpenMP позволяет эффективно распределять вычислительную нагрузку между потоками с помощью директив (`#pragma omp for`). Также OpenMP предоставляет механизмы синхронизации: барьерную (`#pragma omp barrier`) для гарантии завершения работы всех потоков, критическую секцию (`#pragma omp critical`) для защиты данных от одновременного доступа нескольких потоков.

Для установки OpenMP в CentOS Stream необходимо сначала установить компилятор `g++` с помощью команды `dnf install gcc-c++`. Чтобы запустить программу, нужно сначала ее написать. Для этого можно воспользоваться текстовым редактором `nano`, в самом файле

подключается библиотека `<omp.h>`. Для компиляции файла используется команда `g++ -fopenmp filename.cpp filename`, запускается файл по команде `./filename`.

Число одновременно выполняемых потоков можно изменять, используя команду `export OMP_NUM_THREADS=N`, где `N` - количество потоков. В результате работы простейшей программы выводится номер потока (рисунок 8).

```
[root@node1 ~]# export OMP_NUM_THREADS=3
[root@node1 ~]# ./hello
Hello world :: thread 1
Hello world :: thread 0
Hello world :: thread 2
[root@node1 ~]#
```

Рис. 8. Пример работы OpenMP

1.1.5 Настройка OpenMPI

OpenMPI (Message Passing Interface) - это библиотека, предназначенная для разработки параллельных приложений на системах с распределенной памятью. В отличие от OpenMP, который работает с общей памятью, OpenMPI обеспечивает обмен данными между процессами, работающими на различных узлах вычислительного кластера через сетевые коммуникации.

Принцип работы OpenMPI заключается в использовании модели параллелизма SPMD (single program - multiple data). Это означает, что программа запускается в виде независимых процессов на разных узлах. каждый процесс имеет свой ранг и общее с другими процессами коммуникационное пространство, где процессы обмениваются данными через сообщения. Для работы OpenMPI требуется настройка SSH между узлами.

Для установки OpenMPI в CentOS Stream необходимо установить ее на всех узлах командой `dnf install -y openmpi openmpi-devel`. Также для корректной работы нужно добавить пути к исполняемым файлам (PATH) и библиотекам OpenMPI (LD_LIBRARY_PATH). На одном из узлов нужно создать файл `/root/hostfile`, где указывается количество узлов и слотов (слоты - это количество процессов, которые может запустить один узел). После создания файла программы его компилируют командой `mpicc mpi_hello.c -o /root/mpi_hello`.

Чтобы запустить программу, воспользуемся командой `mpirun -hostfile /root/hostfile -np 6 /root/mpi_hello`, где `-np 6` означает общее количество процессов. В случае, если программа не запускается на всех узлах, можно вручную ее скопировать командой `scp /root/mpi_hello node3:/root`. В результате программа `hello_world`

выводит номер потока и имя хоста, на котором она исполнялась (рис. 9).

```
[root@node1 ~]# mpirun --hostfile /root/hostfile -np 6 /root/mpi_hello
Warning: your password will expire in 0 days.
Warning: your password will expire in 0 days.
Hello from process 1 of 6 on host node1
Hello from process 0 of 6 on host node1
Hello from process 5 of 6 on host node3
Hello from process 4 of 6 on host node3
Hello from process 2 of 6 on host node2
Hello from process 3 of 6 on host node2
```

Рис. 9. Пример работы OpenMPI HelloWorld

1.2 Применение технологий параллелизации к исходной программе

В рамках исходной лабораторной работы генерируется случайным образом связный ациклический граф в соответствии с двойным показательным распределением с пятью вершинами. Для заданного графа находится число остовных деревьев. Также строится минимальный по весу остов, полученный остов кодируется и декодируется кодом Прюфера с сохранением весов. На полученном остове реализуется алгоритм нахождения минимального вершинного покрытия.

1.2.1 Применение OpenMP

Для создания параллельных потоков используется директива `#pragma omp parallel`. Она указывает, что структурный блок кода должен быть выполнен параллельно в несколько потоков. Каждый из созданных потоков выполнит одинаковый код содержащийся в блоке, но не одинаковый набор команд. В разных потоках могут выполняться различные ветви или обрабатываться различные данные, что зависит от таких операторов как `if-else` или использования директив распределения работы.

1. Генерация графа

В ходе генерации графа необходимо проверять его на связность. Можно параллельно подсчитывать число изолированных вершин. Для этого используется директива `#pragma omp for reduction(+)`, которая позволяет разделить выполнение итераций цикла между несколькими потоками (`for`), а после выполнения всех операторов параллельной области объединить результаты потоков (`reduction+`). Если больше одной вершины имеют степень равную нулю, то граф несвязен и его нужно перегенерировать.

Для корректного вывода матрицы графа необходимо также использовать редукцию. Потоки независимо обрабатывают части матрицы, а `reduction(+:edge)` потом суммирует количество ребер.

2. Минимальный остов, кодирование и декодирование кодом Прюфера.

Подсчет остовных деревьев реализуется на основе матричной теоремы Кирхгофа. Можно поручить параллельным потокам заполнять минор матрицы с помощью директивы `#pragma omp parallel for collapse(2)`. Она может «сворачивать» вложенные циклы, то есть превращать два цикла в один, где будут сразу две переменные. Это дает более равномерное распараллеливание и балансировку нагрузки.

В заполнении матрицы Кирхгофа можно разделить по потокам различные строки матрицы. Для этого используется директива `#pragma omp parallel for schedule(static)`. Отличие этой директивы состоит в том, что она распределяет итерации цикла между потоками статически на этапе компиляции, равномерно нагружая все потоки.

В вспомогательной функции вычисления определителя параллельно считаются миноры для разных столбцов и с помощью редукции (`reduction(+:det)`) объединяются результаты.

В функции кодирования Прюфера параллелизация используется для подсчета степеней вершин с помощью директив `#pragma omp parallel for` для обработки циклов и `#pragma omp atomic` для безопасного увеличения счетчика степени.

В функции декодирования кодом Прюфера параллелизуется увеличение степеней вершин с использованием атомарных операций на общих данных (`#pragma omp parallel for` и `#pragma omp atomic`).

Алгоритм Прима для нахождения минимального ребра и алгоритм поиска минимального вершинного покрытия нельзя параллелизовать, так как они требуют последовательной обработки. Каждый их шаг зависит от предыдущего, поэтому их можно назвать «жадными».

1.2.2 Применение OpenMPI

В данной программе использовались следующие функции и структуры OpenMPI:

1. `MPI_Init(int* argc, char ** argv)` для инициализации среды выполнения параллельной программы, которая использует сообщения для обмена данными между процессами. Принимает указатели на аргументы командной строки `argc` и `argv` и инициализирует их значения.
2. `MPI_Comm_rank(MPI_Comm comm, int * rank)` для получения ранга (универсального идентификатора) текущего процесса в указанном коммуникаторе (`comm`). Коммуникатор - группа процессов, которые могут обмениваться информацией друг с другом.
3. `MPI_Comm_size(MPI_Comm comm, int* size)` для получения общего количества процессов, которые участвуют в данном коммуникаторе.
4. `MPI_Bcast()` для широковещательной рассылки данных от одного процесса всем остальным
5. `MPI_Send()` для отправки сообщения от одного конкретного процесса другому в указанном коммуникаторе.

6. `MPI_Recv()` для получения сообщения от одного конкретного процесса другому в указанном коммуникаторе. У `Send` и `Recv` должны совпадать тип передаваемых данных, их количество и тег.
7. `MPI_Finalize()` для корректного завершения работы MPI: освобождения ресурсов и состояний. После ее вызова никакие другие функции MPI не могут вызываться.
8. `MPI_Reduce` для объединения элементов определенной операцией (например, суммированием) со всех процессов и сохранения результата на указанном процессе.
9. `MPI_Comm_split` для разделения исходного коммуникатора на несколько подгрупп по определенному правилу.
10. `MPI_Comm_free` для освобождения созданного коммуникатора. Нельзя освобождать встроенные коммуникаторы (`MPI_COMM_WORLD`).

В начале выполнения программы создаются переменные окружения для определения номера текущего процесса `rank` и общего количества процессов `size` в коммуникаторе. В дальнейшем эти переменные передаются во все функции. Вывод осуществляется только в процессе с нулевым рангом для упорядочивания вывода информации.

1. Генерация графа

Для каждого процесса в зависимости от их количества выделяются строки матрицы смежности, при этом остаток распределяется равномерно между первыми процессами. Каждый процесс обрабатывает свой диапазон строк. После этого все процессы отправляют свои строки процессу с рангом 0, который собирает все части в итоговую матрицу графа. Проверка связности выполняется только нулевым процессом, результат проверки рассылается всем процессам. Если граф несвязный, генерация повторяется. В случае успеха генерации финальные матрицы рассылаются всем потокам с помощью функции `MPI_Bcast`.

2. Генерация матрицы Кирхгофа для подсчета количества остовных деревьев

Для каждого процесса выделяются свои строки матрицы, с которыми он работает, вычисляя элементы матрицы Кирхгофа. Затем на нулевом процессе инициализируется глобальная матрица и с помощью `MPI_Reduce` нулевой процесс поэлементно суммирует все полученные от других потоков матрицы и сохраняет его на нулевом процессе, после чего готовую матрицу рассылаем на остальные процессы.

3. Код Прюфера и минимальное вершинное покрытие

Так как кодирование и декодирование кодом Прюфера, а также нахождение минимального вершинного покрытия являются итерационными алгоритмами, их нельзя распараллелить. Однако они могут выполняться независимо друг от друга. Для этого можно разделить процессы на две группы и создать подкоммуникатор с помощью команды `MPI_Comm_split`. Далее задачи выполняются в разных группах, при этом в функции передаются обновленные переменные количества процессов и текущего процесса. Только нулевой процесс в этом коммуникаторе выводит результаты. После выполнения всех процессов освобождаются процессы, связанные с созданным коммуникатором с помощью команды `MPI_Comm_free`.

2 Результаты работы программы

При запуске программы выводятся сообщение о номере процесса и результаты функций, которые в этом процессе выполняются.

```
Graph generated!
Vertices: 5, edges: 10
  | A | B | C | D | E |
-----
A | 0 | 1 | 4 | 2 | 2 |
-----
B | 1 | 0 | 3 | 2 | 2 |
-----
C | 4 | 3 | 0 | 2 | 4 |
-----
D | 2 | 2 | 2 | 0 | 2 |
-----
E | 2 | 2 | 4 | 2 | 0 |
-----
Kirchoffs matix is generated

Count of spanning trees = 125
Weight of min spanning tree: 7

Constructing MST
Vertices: 5, edges: 4
  | A | B | C | D | E |
-----
A | 0 | 1 | 0 | 2 | 2 |
-----
B | 1 | 0 | 0 | 0 | 0 |
-----
C | 0 | 0 | 0 | 2 | 0 |
-----
D | 2 | 0 | 2 | 0 | 0 |
-----
E | 2 | 0 | 0 | 0 | 0 |
-----
Vertex cover:
A C
Prufer code: Prufer code: [ 0 3 0 4 ]
Edge weights: [ 1 2 2 2 ]
Decoded MST:
Vertices:5, edges: 4
  | A | B | C | D | E |
-----
A | 0 | 1 | 0 | 2 | 2 |
-----
B | 1 | 0 | 0 | 0 | 0 |
-----
C | 0 | 0 | 0 | 2 | 0 |
-----
D | 2 | 0 | 2 | 0 | 0 |
-----
E | 2 | 0 | 0 | 0 | 0 |
-----
Success: Original and decoded MST match!
```

Рис. 10. Выполнение программы

Заключение

В ходе выполнения научно-исследовательской работы были освоены основы работы с дистрибутивом CentOS Stream на базе ядра Linux, проведена установка и настройка среды виртуализации VirtualBox. Также созданы три виртуальные машины, между которыми налажена NAT-сеть и SSH-подключение. В рамках основной части работы были изучены технологии параллельного программирования OpenMP и OpenMPI, адаптирована под многопоточное и многопроцессорное выполнение ранее написанная лабораторная работа по теории графов.

Список использованной литературы

- [1] Знакомство с OpenMP // Geeksforgeeks URL: <https://www.geeksforgeeks.org/openmp-hello-world-program/> (дата обращения 20.06.2025)
- [2] Параллельные заметки №3 - базовые конструкции OpenMP // Хабр URL: <https://habr.com/ru/companies/intel/articles/85273/> (дата обращения 21.06.2025)
- [3] MPI - введение и первая программа // Хабр URL: <https://habr.com/ru/articles/548266/> (дата обращения 21.06.2025)
- [4] MPI для начинающих // OpenNET URL: www.opennet.ru/docs/RUS/MPI_intro/ (дата обращения 23.06.2025)
- [5] Документация CentOS Stream // Centos URL: <https://www.centos.org/centos-stream/> (дата обращения 20.06.2025)

Приложение А «Исходный код программы: файл main.cpp»

Листинг 1. Файл main.cpp

```
1  #include <iostream>
2  #include <string>
3  #include <chrono>
4  #include <iomanip>
5  #include <mpi.h>
6  #include <omp.h>
7
8  #include "graph.h"
9
10
11 bool pos = true;
12
13 int main(int argc, char** argv) {
14     MPI_Init(&argc, &argv);
15     int rank, size;
16     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
17     MPI_Comm_size(MPI_COMM_WORLD, &size);
18     int v = 5;
19     Graph graph(v);
20     int group;
21
22     std::mt19937 gen(std::random_device{}());
23     graph.generateGraph(gen, size, rank);
24     if (rank == 0) {
25         std::cout << "Graph generated!" << std::endl;
26         graph.printMatrix(graph.unoriented);
27     }
28
29     graph.generateKirchhoff(rank, size);
30     if (rank == 0) {
31         printf("Kirchoffs matix is generated\n");
32     }
33
34     int count = graph.countSpanTree(rank);
35     if (rank == 0) {
36         cout << "\nCount of spanning trees = " << count << std::endl;
37     }
38
39     if (rank == 0) {
40         graph.mst = graph.Prim(rank);
41     }
42     else {
43         graph.mst.assign(v, vector<int>(v, 0));
44     }
45     graph.broadcastMST(graph.mst, rank);
46
47     if (rank == 0) {
48         std::cout << "\nConstructing MST" << std::endl;
49         graph.printMatrix(graph.mst);
50     }
51     if (size > 1) {
52         group = (rank < size / 2) ? 0 : 1;
53     }
54     else {
```

```

55     group = 0;
56 }
57
58 MPI_Comm subComm;
59 MPI_Comm_split(MPI_COMM_WORLD, group, rank, &subComm);
60 int subRank, subSize;
61 MPI_Comm_rank(subComm, &subRank);
62 MPI_Comm_size(subComm, &subSize);
63
64 if (group == 0) {
65     auto prufer = graph.pruferEncode(graph.mst, subRank, subSize,
66     subComm);
67     auto decodedMST = graph.pruferDecode(prufer.first, prufer.second,
68     subRank, subSize, subComm);
69
70     if (subRank == 0) {
71         std::cout << "Prufer code: ";
72         graph.printPruferCode(prufer);
73         std::cout << "Decoded MST:\n";
74         graph.printMatrix(decodedMST);
75         if (graph.mst == decodedMST) {
76             cout << "Success: Original and decoded MST match!" << endl;
77         }
78         else {
79             cout << "Error: Original and decoded MST differ!" << endl;
80         }
81     }
82     if (size == 1) {
83         MPI_Comm_free(&subComm);
84         group = 1;
85         MPI_Comm_split(MPI_COMM_WORLD, group, rank, &subComm);
86         MPI_Comm_rank(subComm, &subRank);
87         MPI_Comm_size(subComm, &subSize);
88         auto cover = graph.findMinCover();
89         std::cout << "Vertex cover:\n";
90         for (int x : cover) std::cout << char('A' + x) << " ";
91         std::cout << std::endl;
92     }
93 }
94 else {
95     auto cover = graph.findMinCover();
96     if (subRank == 0) {
97         std::cout << "Vertex cover:\n";
98         for (int x : cover) std::cout << char('A' + x) << " ";
99         std::cout << std::endl;
100     }
101 }
102
103 MPI_Comm_free(&subComm);
104
105 MPI_Finalize();
106 return 0;
107 }

```

Приложение Б «Исходный код программы: файл graph.cpp»

Листинг 2. Файл graph.cpp

```
1  #include "graph.h"
2  #include <queue>
3  #include <cassert>
4  #include <string>
5  #include <set>
6  #include <map>
7  #include <unordered_set>
8  #include <stack>
9  #include <fstream>
10 #include <mpi.h>
11 #include <omp.h>
12
13 Graph::Graph(int vert) : v(vert), adjMatrix(vert, vector<int>(vert, 0))
14 {}
15
16 int Graph::generateDoubleExponential(mt19937& gen, int n, bool bounded,
17 bool isPositive) {
18     random_device rd;
19     uniform_real_distribution<> dis(0.0001, 0.9999);
20     double lambda = 0.6;
21     double mu = max(1.0, n / 2.0);
22     double x_i;
23     do {
24         double r_i = dis(gen);
25         x_i = (1.0 / lambda) * (log(mu) - log(-log(r_i)));
26     }
27     while (abs(x_i) < 1 or (bounded and (x_i < 1 || x_i > n)));
28     int result = static_cast<int>(round(x_i));
29     if (abs(result) > 1000) {
30         result = 2;
31     }
32     return result;
33 }
34
35 vector<size_t> Graph::out_degr(size_t vert, mt19937& gen) {
36     if (vert == 0) { return {}; }
37     else if (vert == 1) { return {0}; }
38     else if (vert == 2) {
39         vector<size_t> res = { 1, 1 };
40         return res;
41     }
42     else {
43         vector<size_t> res(vert);
44         int odd = 0;
45         for (size_t i = 0; i < vert; ++i) {
46             int outDegree = generateDoubleExponential(gen, vert-1, true, true);
47             ;
48             res[i] = outDegree;
49             if (outDegree % 2 != 0) { odd++; }
50         }
51         if (odd % 2 != 0) {
52             for (int i = 0; i < vert; ++i) {
53                 if (res[i] % 2 != 0 && res[i] < vert - 1) {
```

```

52         res[i]++;
53         odd--;
54         break;
55     }
56 }
57 }
58 sort(res.rbegin(), res.rend());
59 for (size_t i = 0; i < vert; ++i) {
60     res[i] = min(vert - i - 1, res[i]);
61 }
62 odd = 0;
63 for (size_t i = 0; i < vert; ++i) {
64     if (res[i] % 2 != 0) { odd++; }
65 }
66 if (odd % 2 != 0) {
67     if (res.back() < vert - 1) {
68         res.back()++;
69     }
70 }
71 return res;
72 }
73 }
74
75 bool Graph::isConnected() {
76     if (v < 2) return true;
77     int zeroCount = 0;
78
79     #pragma omp parallel for reduction(+:zeroCount)
80     for (int i = 0; i < v; ++i) {
81         int degree = 0;
82         for (int j = 0; j < v; ++j) {
83             degree += adjMatrix[i][j];
84         }
85         if (degree == 0) {
86             zeroCount++;
87         }
88     }
89     return zeroCount <= 1;
90 }
91
92 void Graph::generateGraph(mt19937& gen, int size, int rank) {
93     q = 0;
94     if (v < 2) {
95         adjMatrix.assign(v, vector<int>(v, 0));
96         unoriented.assign(v, vector<int>(v, 0));
97         return;
98     }
99     int rowsPerProc = v / size; //строки по процессам
100     int extra = v % size;
101     int startRow = rank * rowsPerProc + std::min(rank, extra);
102     int endRow = startRow + rowsPerProc + (rank < extra ? 1 : 0);
103     int localRows = endRow - startRow;
104     bool connected = false;
105     do {
106         vector<vector<int>> localAdj(localRows, vector<int>(v, 0)); //
    локальные подматрицы
107         vector<vector<int>> localUnor(localRows, vector<int>(v, 0));
108         auto degr = out_degr(v, gen);
109         for (int i = 0; i < localRows; ++i) {
110             int global_i = startRow + i;

```



```

111     for (int j = global_i + 1; j < std::min(global_i + 1 + static_cast
<int>(degr[global_i]), v); ++j) {
112         int val = generateDoubleExponential(gen, v, false, true);
113         localAdj[i][j] = val;
114         localUnor[i][j] = val;
115     }
116 }
117 if (rank == 0) { //сбор всех локальных
118     adjMatrix.assign(v, vector<int>(v, 0));
119     unoriented.assign(v, vector<int>(v, 0));
120     for (int i = 0; i < localRows; ++i) {
121         adjMatrix[startRow + i] = localAdj[i];
122         unoriented[startRow + i] = localUnor[i];
123     }
124     for (int src = 1; src < size; ++src) {
125         int sRow = src * rowsPerProc + std::min(src, extra);
126         int eRow = sRow + rowsPerProc + (src < extra ? 1 : 0);
127         int rows = eRow - sRow;
128         vector<int> buffer(v * rows);
129         MPI_Recv(buffer.data(), buffer.size(), MPI_INT, src, 0,
MPI_COMM_WORLD, MPI_STATUS_IGNORE);
130         for (int i = 0; i < rows; ++i)
131             adjMatrix[sRow + i] = vector<int>(buffer.begin() + i * v, buffer
.begin() + (i + 1) * v);
132         MPI_Recv(buffer.data(), buffer.size(), MPI_INT, src, 1,
MPI_COMM_WORLD, MPI_STATUS_IGNORE);
133         for (int i = 0; i < rows; ++i)
134             unoriented[sRow + i] = vector<int>(buffer.begin() + i * v,
buffer.begin() + (i + 1) * v);
135     }
136     for (int i = 0; i < v; ++i) { //симметрично заполняем нижнетреугольную
137         for (int j = i + 1; j < v; ++j) {
138             if (unoriented[i][j] != 0)
139                 unoriented[j][i] = unoriented[i][j];
140             else if (unoriented[j][i] != 0)
141                 unoriented[i][j] = unoriented[j][i];
142         }
143     }
144 }
145 else {
146     vector<int> flatAdj, flatUnor;
147     for (const auto& row : localAdj)
148         flatAdj.insert(flatAdj.end(), row.begin(), row.end());
149     for (const auto& row : localUnor)
150         flatUnor.insert(flatUnor.end(), row.begin(), row.end());
151
152     MPI_Send(flatAdj.data(), flatAdj.size(), MPI_INT, 0, 0,
MPI_COMM_WORLD);
153     MPI_Send(flatUnor.data(), flatUnor.size(), MPI_INT, 0, 1,
MPI_COMM_WORLD);
154 }
155 if (rank == 0)
156     connected = isConnected();
157 MPI_Bcast(&connected, 1, MPI_C_BOOL, 0, MPI_COMM_WORLD);
158 } while (!connected);
159 if (rank == 0) { //принимает финальные матрицы
160     vector<int> flatAdj, flatUnor;
161     for (auto& row : adjMatrix) flatAdj.insert(flatAdj.end(), row.begin
()), row.end());
162     for (auto& row : unoriented) flatUnor.insert(flatUnor.end(), row.

```

```

163 begin(), row.end());
164 MPI_Bcast(flatAdj.data(), v * v, MPI_INT, 0, MPI_COMM_WORLD);
165 MPI_Bcast(flatUnor.data(), v * v, MPI_INT, 0, MPI_COMM_WORLD);
166 }
167 else {
168     vector<int> flatAdj(v * v), flatUnor(v * v);
169     MPI_Bcast(flatAdj.data(), v * v, MPI_INT, 0, MPI_COMM_WORLD);
170     MPI_Bcast(flatUnor.data(), v * v, MPI_INT, 0, MPI_COMM_WORLD);
171
172     adjMatrix.assign(v, vector<int>(v));
173     unoriented.assign(v, vector<int>(v));
174     for (int i = 0; i < v; ++i) {
175         for (int j = 0; j < v; ++j) {
176             adjMatrix[i][j] = flatAdj[i * v + j];
177             unoriented[i][j] = flatUnor[i * v + j];
178         }
179     }
180 }
181 }
182
183
184
185
186 void Graph::printMatrix(const vector<vector<int>>& matrix) {
187     int edge = 0;
188     #pragma omp parallel for reduction(+:edge)
189     for (int i = 0; i < v; ++i) {
190         for (int j = i; j < v; ++j) {
191             if (matrix[i][j] != 0) { edge++; }
192         }
193     }
194     cout << "Vertices: " << v << ", edges: " << edge << endl;
195     if (v == 0) {
196         cout << "Empty" << endl;
197         return;
198     }
199     vector<int> max_len(v, 1);
200     for (int i = 0; i < v; ++i) {
201         for (int j = 0; j < v; ++j) {
202             int len = to_string(matrix[i][j]).length();
203             if (len > max_len[j]) {
204                 max_len[j] = len;
205             }
206         }
207     }
208     cout << "    |";
209     for (int j = 0; j < v; ++j) {
210         cout << " " << (char)('A' + j);
211         for (int k = 0; k < max_len[j] - 1; ++k) cout << " ";
212         cout << " |";
213     }
214     cout << "\n";
215     cout << "  -";
216     for (int j = 0; j < v; ++j) {
217         for (int k = 0; k < max_len[j] + 2; ++k) cout << "-";
218         cout << "-";
219     }
220     cout << endl;
221     for (int i = 0; i < v; ++i) {

```

```

222     cout << " " << (char)('A' + i) << " |";
223     for (int j = 0; j < v; ++j) {
224         cout << " " << matrix[i][j];
225         int len = to_string(matrix[i][j]).length();
226         for (int k = 0; k < max_len[j] - len; ++k) cout << " ";
227         cout << " |";
228     }
229     cout << "\n";
230     cout << " -";
231     for (int j = 0; j < v; ++j) {
232         for (int k = 0; k < max_len[j] + 2; ++k) cout << "-";
233         cout << "-";
234     }
235     cout << endl;
236 }
237 }
238
239
240
241 void Graph::generateKirchhoff(int rank, int size) {
242     vector<vector<int>> localK(v, vector<int>(v, 0));
243
244     #pragma omp parallel for schedule(static)
245     for (int i = rank; i < v; i += size) {
246         int degree = 0;
247         for (int j = 0; j < v; ++j) {
248             if (unoriented[i][j] != 0) degree++;
249         }
250         localK[i][i] = degree;
251         for (int j = i + 1; j < v; ++j) {
252             if (unoriented[i][j] != 0) {
253                 localK[i][j] = localK[j][i] = -1;
254             }
255         }
256     }
257     if (rank == 0) {
258         kirchhoff.assign(v, vector<int>(v, 0));
259     }
260     vector<int> flatLocalK;
261     for (auto& row : localK)
262         flatLocalK.insert(flatLocalK.end(), row.begin(), row.end());
263
264     vector<int> flatGlobalK(v * v);
265     MPI_Reduce(flatLocalK.data(), flatGlobalK.data(), v * v, MPI_INT,
266               MPI_SUM, 0, MPI_COMM_WORLD);
267
268     if (rank == 0) {
269         for (int i = 0; i < v; ++i)
270             kirchhoff[i] = vector<int>(flatGlobalK.begin() + i * v, flatGlobalK.
271             begin() + (i + 1) * v);
272     }
273     vector<int> fullFlatK(v * v);
274     if (rank == 0) fullFlatK = flatGlobalK;
275     MPI_Bcast(fullFlatK.data(), v * v, MPI_INT, 0, MPI_COMM_WORLD);
276
277     if (rank != 0) {
278         kirchhoff.assign(v, vector<int>(v));
279         for (int i = 0; i < v; ++i)
280             for (int j = 0; j < v; ++j)
281                 kirchhoff[i][j] = fullFlatK[i * v + j];

```

```

280     }
281 }
282
283 int Graph::countSpanTree(int rank) {
284     if (rank != 0) return 0;
285
286     if (v == 0) return 0;
287     if (v == 1) return 1;
288
289     vector<vector<int>> minor(v - 1, vector<int>(v - 1));
290
291     #pragma omp parallel for collapse(2)
292     for (int i = 1; i < v; ++i) {
293         for (int j = 1; j < v; ++j) {
294             minor[i - 1][j - 1] = kirchhoff[i][j];
295         }
296     }
297
298     return determinant(minor);
299 }
300
301 int Graph::determinant(vector<vector<int>> m) {
302     int vm = m.size();
303     if (vm == 1) { return m[0][0]; }
304     else if (vm == 2) {
305         return m[0][0] * m[1][1] - m[0][1] * m[1][0];
306     }
307     int det = 0;
308     #pragma omp parallel for reduction(+:det)
309     for (int col = 0; col < vm; ++col) {
310         vector<vector<int>> minor(vm - 1, vector<int>(vm - 1));
311
312         for (int i = 1; i < vm; ++i) {
313             int mcol = 0;
314             for (int j = 0; j < vm; ++j) {
315                 if (j == col) continue;
316                 minor[i - 1][mcol++] = m[i][j];
317             }
318         }
319
320         int sign = (col % 2 == 0) ? 1 : -1;
321         det += sign * m[0][col] * determinant(minor);
322     }
323
324     return det;
325 }
326
327 vector<vector<int>> Graph::Prim(int rank) {
328     mst.assign(v, vector<int>(v, 0)); //adjMatrix spanning tree
329     set<int> S;
330     int u = 0;
331     S.insert(u);
332     vector<int> alpha(v, -1);
333     vector<int> beta(v, INF);
334
335
336     for (int v = 0; v < this->v; ++v) {
337         if (unoriented[u][v] != 0){
338             alpha[v] = u;
339             beta[v] = unoriented[u][v];

```

```

340     }
341     else {
342         alpha[v] = 0;
343         beta[v] = INF;
344     }
345 }
346
347 for (int i = 1; i < v; i++) {
348     int min_beta = INF;
349     int w = -1;
350     for (int v = 0; v < this->v; ++v) {
351         if (S.count(v) == 0 and beta[v] < min_beta) {
352             min_beta = beta[v];
353             w = v;
354         }
355     }
356     if (w == -1) break;
357     S.insert(w);
358     mst[alpha[w]][w] = beta[w];
359     mst[w][alpha[w]] = beta[w];
360
361     for (int v = 0; v < this->v; ++v) {
362         if (unoriented[w][v] != 0 and S.count(v) == 0) {
363             if (unoriented[w][v] < beta[v]) {
364                 alpha[v] = w;
365                 beta[v] = unoriented[w][v];
366             }
367         }
368     }
369 }
370 int totalWeight = 0;
371 for (int i = 0; i < v; ++i) {
372     for (int j = i + 1; j < v; ++j) {
373         totalWeight += mst[i][j];
374     }
375 }
376 if (rank == 0) {
377     cout << "Weight of min spanning tree: " << totalWeight << endl;
378 }
379
380 return mst;
381 }
382
383 pair<vector<int>, vector<int>> Graph::pruferEncode(const vector<vector<
384 int>>& mst, int rank, int size, MPI_Comm comm) {
385     vector<int> pruferCode;
386     vector<int> weights;
387     vector<int> degree(v, 0);
388     set<int> leaves;
389
390     #pragma omp parallel for
391     for (int i = 0; i < v; ++i) {
392         for (int j = 0; j < v; ++j) {
393             if (mst[i][j] != 0) {
394                 #pragma omp atomic
395                 degree[i]++;
396             }
397         }
398     }

```

```

399     for (int i = 0; i < v; ++i) {
400         if (degree[i] == 1) {
401             leaves.insert(i);
402         }
403     }
404
405     vector<vector<int>> tempMST = mst;
406     while (pruferCode.size() < v - 1) {
407         int leaf = *leaves.begin();
408         leaves.erase(leaf);
409         int neighbor = -1;
410         for (int i = 0; i < v; ++i) {
411             if (tempMST[leaf][i] != 0) {
412                 neighbor = i;
413                 break;
414             }
415         }
416         pruferCode.push_back(neighbor);
417         weights.push_back(tempMST[leaf][neighbor]);
418         tempMST[leaf][neighbor] = 0;
419         tempMST[neighbor][leaf] = 0;
420         degree[neighbor]--;
421         if (degree[neighbor] == 1) {
422             leaves.insert(neighbor);
423         }
424     }
425
426     if (leaves.size() == 2) {
427         auto it = leaves.begin();
428         int u = *it++;
429         int w = *it;
430         weights.push_back(mst[u][w]);
431     }
432
433     int codeSize = pruferCode.size();
434     MPI_Bcast(&codeSize, 1, MPI_INT, 0, comm);
435     if (rank != 0) pruferCode.resize(codeSize);
436     MPI_Bcast(pruferCode.data(), codeSize, MPI_INT, 0, comm);
437
438     int weightSize = weights.size();
439     MPI_Bcast(&weightSize, 1, MPI_INT, 0, comm);
440     if (rank != 0) weights.resize(weightSize);
441     MPI_Bcast(weights.data(), weightSize, MPI_INT, 0, comm);
442
443
444     return { pruferCode, weights };
445 }
446
447 vector<vector<int>> Graph::pruferDecode(const vector<int>& pruferCode,
448     const vector<int>& weights, int rank, int size, MPI_Comm comm) {
449     vector<vector<int>> decodedMST(v, vector<int>(v, 0));
450
451     vector<int> localCode = pruferCode;
452     vector<int> localWeights = weights;
453     if (rank == 0) {
454         vector<int> degree(v, 1);
455         set<int> leaves;
456
457         #pragma omp parallel for

```

```

458     for (int i = 0; i < pruferCode.size(); ++i) {
459         #pragma omp atomic
460         degree[pruferCode[i]]++;
461     }
462
463     for (int i = 0; i < v; ++i) {
464         if (degree[i] == 1) {
465             leaves.insert(i);
466         }
467     }
468
469     for (int i = 0; i < pruferCode.size(); ++i) {
470         int leaf = *leaves.begin();
471         leaves.erase(leaf);
472         int parent = pruferCode[i];
473         decodedMST[leaf][parent] = weights[i];
474         decodedMST[parent][leaf] = weights[i];
475
476         degree[parent]--;
477         if (degree[parent] == 1) {
478             leaves.insert(parent);
479         }
480     }
481
482     if (leaves.size() == 2) {
483         auto it = leaves.begin();
484         int u = *it++;
485         int w = *it;
486         decodedMST[u][w] = weights.back();
487         decodedMST[w][u] = weights.back();
488     }
489
490 }
491
492 for (int i = 0; i < v; ++i) {
493     MPI_Bcast(decodedMST[i].data(), v, MPI_INT, 0, comm);
494 }
495 return decodedMST;
496 }
497
498 vector<int> Graph::findMinCover() { //вершины должны покрывать вершины
499     vector<int> cover;
500     vector<vector<int>> graph = mst ;
501     vector<bool> removed(v, false);
502
503     while (true) {
504         double best = -1.0;
505         int u = -1;
506         for (int i = 0; i < v; ++i) {
507             if (removed[i]) continue;
508             int degree = 0;
509             int totalWeight = 0;
510             for (int j = 0; j < v; ++j) {
511                 if (!removed[j] and graph[i][j] != 0) {
512                     degree++;
513                     totalWeight += graph[i][j];
514                 }
515             }
516             if (degree > best) {
517                 best = degree;
518                 u = i;

```

```
518     }
519   }
520   if (u < 0) break;
521   cover.push_back(u);
522   removed[u] = true;
523   for (int j = 0; j < v; ++j) {
524     if (graph[u][j] != 0) {
525       removed[j] = true;
526     }
527   }
528 }
529 return cover;
530 }
531
```


Приложение В «Исходный код программы: файл graph.h»

Листинг 3. Файл graph.h

```
1  #pragma once
2
3  #include <iostream>
4  #include <vector>
5  #include <random>
6  #include <algorithm>
7  #include <functional>
8  #include <cmath>
9  #include <climits>
10 #include <mpi.h>
11 #include <omp.h>
12
13 #define INF INT_MAX
14
15 using namespace std;
16 extern bool pos;
17
18 class Graph
19 {
20     private:
21         int v;
22         int q;
23         int generateDoubleExponential(mt19937& gen, int n, bool bounded, bool
isPositive);
24         vector<size_t> out_degr(size_t vert, mt19937& gen);
25         bool isConnected();
26     public:
27         vector<vector<int>> adjMatrix;
28         vector<vector<int>> weightMatrix;
29         vector<vector<int>> residual;
30         vector<vector<int>> kirchhoff;
31         vector<vector<int>> unoriented;
32         vector<vector<int>> mst;
33         Graph(int v);
34         void generateGraph(mt19937& gen, int rank, int size);
35         void printMatrix(const vector<vector<int>>& matrix);
36         const int getV() { return v; }
37         const int getQ() { return q; }
38         void generateKirchhoff(int rank, int size);
39         int countSpanTree(int rank);
40         int determinant(vector<vector<int>> minor);
41         vector<vector<int>> Prim(int rank);
42         pair<vector<int>, vector<int>> pruferEncode(const vector<vector<int>>&
mst, int world_rank, int world_size, MPI_Comm comm);
43         vector<vector<int>> pruferDecode(const vector<int>& pruferCode, const
vector<int>& weights, int world_rank, int world_size, MPI_Comm comm);
44         void printPruferCode(const pair<vector<int>, vector<int>>& code) {
45             cout << "Prufer code: [";
46             for (int node : code.first) {
47                 cout << " " << node;
48             }
49             cout << " ]" << endl;
50             cout << "Edge weights: [";
51             for (int weight : code.second) {
```

```

52     cout << " " << weight;
53 }
54 cout << " ]" << endl;
55 }
56 vector<int> findMinCover();
57 void broadcastMST(vector<vector<int>>& mst, int rank) {
58     int n;
59
60     if (rank == 0) {
61         n = mst.size();
62     }
63     MPI_Bcast(&n, 1, MPI_INT, 0, MPI_COMM_WORLD);
64
65     std::vector<int> flatMST(n * n);
66     if (rank == 0) {
67         for (int i = 0; i < n; ++i)
68             for (int j = 0; j < n; ++j)
69                 flatMST[i * n + j] = mst[i][j];
70     }
71
72     MPI_Bcast(flatMST.data(), n * n, MPI_INT, 0, MPI_COMM_WORLD);
73     if (rank != 0) {
74         mst.resize(n, std::vector<int>(n));
75         for (int i = 0; i < n; ++i)
76             for (int j = 0; j < n; ++j)
77                 mst[i][j] = flatMST[i * n + j];
78     }
79 }
80 };
81
82

```